Notes Techniques — VoxPlace

> **Version ciblée :** OpenGL 4.6 Core (`#version 460 core`) Ce document rassemble toutes les notions essentielles du projet : bases OpenGL, pipeline de rendu voxel, optimisations GPU/CPU, architecture réseau, et décisions de design.

Commande Valgrind :

```
valgrind -s --leak-check=full --show-leak-kinds=all --track-origins=yes
./build/VoxPlace > valgrind.txt 2>&1
```

Sommaire

1. Bases OpenGL
2. Pipeline Technique : Vertex Pulling & SSBO
3. État Actuel de l'Implémentation
4. Optimisations GPU (Pipeline de Rendu)
5. Optimisations CPU (Types, Cache & Mémoire)
6. Multithreading (CPU)
7. Architecture Serveur & Réseau
8. Architecture VoxPlace : Lazy Generation
9. Rendu Dynamique & UX des Bordures
10. Les "Fausses" Bonnes Idées (À Éviter)
11. To-Do
12. Liens Utiles
13. Bonnes Pratiques OpenGL

---

1. Bases OpenGL

Principes de base

- OpenGL fonctionne en mode **état-machine** : on configure des états (blend, cull, depth test), on lie des buffers et on envoie des commandes de dessin.
- Le **Programmable Pipeline** (shaders obligatoires) est le standard depuis OpenGL 3.3+.
- VoxPlace cible **OpenGL 4.6 Core** (requis pour les SSBO, `#version 460 core` dans les shaders).

Double Buffering

- Évite le flickering en dessinant sur un **back buffer** (off-screen), puis en swapant les buffers pour afficher l'image complète.
- `glfwSwapBuffers(window)` effectue le swap.

Pipeline de Rendu (Simplifié)

Étape	Rôle
1. **Vertex Shader**	Transforme les positions, calcule UV/normales
2. **Shape Assembly**	Assemble les primitives à partir des sommets
3. **Geometry Shader**	*(Optionnel)* Génère/transforme des primitives
4. **Rasterisation**	Convertit les primitives en fragments/pixels
5. **Fragment Shader**	Calcule la couleur finale, lighting
6. **Tests & Blending**	Depth test, stencil test, blending

> Les deux étapes critiques sont le **Vertex Shader** et le **Fragment Shader**.

Objets OpenGL

Objet	Rôle
VBO (Vertex Buffer Object)	Buffer mémoire GPU contenant les attributs (positions, normales, UV)
VAO (Vertex Array Object)	Décrit le layout des attributs et lie les VBO
EBO (Element Buffer Object)	Indices pour dessiner via `glDrawElements`
SSBO (Shader Storage Buffer)	Buffer flexible, accessible comme un tableau C++ dans le shader *(OpenGL 4.3+)* — **utilisé par VoxPlace**

```
// Exemple : VBO + VAO minimal
float vertices[] = {
    -0.5f, -0.5f, 0.0f,
     0.5f, -0.5f, 0.0f,
     0.0f,  0.5f, 0.0f
};
unsigned int VBO, VAO;
glGenVertexArrays(1, &VAO);
glGenBuffers(1, &VBO);
glBindVertexArray(VAO);
glBindBuffer(GL_ARRAY_BUFFER, VBO);
glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float),
(void*)0);
glEnableVertexAttribArray(0);
glBindBuffer(GL_ARRAY_BUFFER, 0);
glBindVertexArray(0);
```

```
// Exemple : EBO (Index Buffer)
float vertices[] = {
     0.5f,  0.5f, 0.0f,   // 0 : top right
     0.5f, -0.5f, 0.0f,   // 1 : bottom right
    -0.5f, -0.5f, 0.0f,   // 2 : bottom left
    -0.5f,  0.5f, 0.0f    // 3 : top left
};
unsigned int indices[] = {
    0, 1, 3,   // premier triangle
    1, 2, 3    // second triangle
};
```

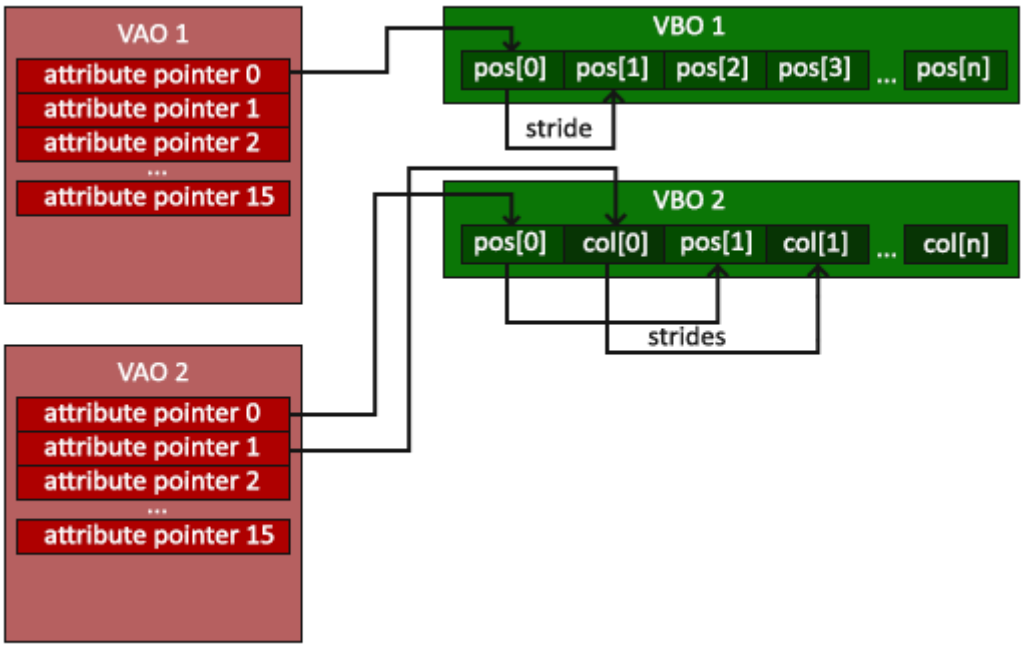

Figure : Organisation d'un VAO et ses VBO/EBO associés.

Variables Uniformes

Passage de données constantes du programme C++ vers les shaders.

```
unsigned int shaderProgram = ...; // Programme shader compilé
glUseProgram(shaderProgram);
int uniformLocation = glGetUniformLocation(shaderProgram, "myUniform");
glUniform1f(uniformLocation, 0.5f);
```

---

2. Pipeline Technique : Vertex Pulling & SSBO

Concept : Inversion du Contrôle

Le **Vertex Pulling** lit directement le SSBO via `gl_VertexID`. On ne passe pas de VBO, pas de Vertex Attributes. Le shader **"tire" (pull)** les données du SSBO lui-même.

	Pipeline Classique (`Chunk.h`)	**Vertex Pulling (`Chunk2.h`)**
Données envoyées	4 sommets/face → ~104 octets	1 `uint32_t`/face → **4 octets**
Réduction mémoire	—	**×25**
Qui génère les sommets ?	Le CPU	Le GPU (Vertex Shader)

> **Note :** VoxPlace conserve `Chunk.h` (ancienne implémentation avec VBO classique) et `Chunk2.h` (implémentation actuelle avec Vertex Pulling + SSBO). Le pipeline actif est `Chunk2`.

Côté CPU : Bit Packing (Layout réel de `Chunk2.h`)

On compresse toutes les infos d'une face + AO dans un seul `uint32_t` :

Champ	Bits	Masque	Plage	Shift
X local	`[0–3]` — 4 bits	`0xF`	0–15	`<< 0`
Y local	`[4–11]` — 8 bits	`0xFF`	0–255	`<< 4`
Z local	`[12–15]` — 4 bits	`0xF`	0–15	`<< 12`
Face Direction	`[16–18]` — 3 bits	`0x7`	0–5	`<< 16`
Color Index	`[19–23]` — 5 bits	`0x1F`	0–31	`<< 19`
AO vertex 0	`[24–25]` — 2 bits	`0x3`	0–3	`<< 24`
AO vertex 1	`[26–27]` — 2 bits	`0x3`	0–3	`<< 26`
AO vertex 2	`[28–29]` — 2 bits	`0x3`	0–3	`<< 28`
AO vertex 3	`[30–31]` — 2 bits	`0x3`	0–3	`<< 30`

Total utilisé : 32/32 bits — Color est packé comme `color-1` (air = 0, jamais rendu, donc offset de 1 est safe)

```
// Bit packing réel (Chunk2.h, meshGenerate)
uint32_t packed = x | (y << 4) | (z << 12) | (faceDir << 16)
    | ((color - 1) << 19) | (ao0 << 24) | (ao1 << 26) | (ao2 << 28) | (ao3
<< 30);
```

> **Performance `std::vector`** : Sans `.reserve()`, chaque `push_back` qui dépasse la capacité déclenche une réallocation : `malloc()` → `memcpy()` → `free()`. Toujours appeler `.reserve(estimatedSize)` avant de remplir le vecteur. La ligne `packedFaces.reserve(...)` est actuellement **commentée** dans le code.

Côté CPU : Envoi & Dessin (extrait de `Chunk2.h`)

```
// uploadMesh() — Crée/recrée le SSBO
glGenBuffers(1, &ssbo);
glBindBuffer(GL_SHADER_STORAGE_BUFFER, ssbo);
glBufferData(GL_SHADER_STORAGE_BUFFER,
             faces.size() * sizeof(uint32_t),
             faces.data(), GL_STATIC_DRAW);

// render() — Dessin chaque frame
glBindBufferBase(GL_SHADER_STORAGE_BUFFER, 0, ssbo); // Binding = 0
glBindVertexArray(vao);  // VAO dummy (obligatoire mais vide)
glDrawArrays(GL_TRIANGLES, 0, faceCount * 6);  // 1 face = 6 sommets (2
triangles)
```

Côté GPU : Vertex Shader (`chunk2.vs`)

```
#version 460 core

layout(std430, binding = 0) readonly buffer ChunkData {
    uint faces[];
};

uniform mat4 view;
uniform mat4 projection;
uniform vec3 chunkPos;

out flat int vColorIndex;
out flat int vFaceDir;
out vec3 vFragPos;

// Lookup tables (pas de if/else)
const int QUAD_INDICES[6] = int[6](0, 1, 2, 0, 2, 3);

const vec3 FACE_OFFSETS[24] = vec3[24](
    // Face 0: TOP (+Y)
    vec3(0, 1, 0), vec3(1, 1, 0), vec3(1, 1, 1), vec3(0, 1, 1),
    // Face 1: BOTTOM (-Y)
    vec3(0, 0, 1), vec3(1, 0, 1), vec3(1, 0, 0), vec3(0, 0, 0),
    // Face 2: NORTH (+Z)
    vec3(0, 0, 1), vec3(0, 1, 1), vec3(1, 1, 1), vec3(1, 0, 1),
    // Face 3: SOUTH (-Z)
    vec3(1, 0, 0), vec3(1, 1, 0), vec3(0, 1, 0), vec3(0, 0, 0),
    // Face 4: EAST (+X)
    vec3(1, 0, 1), vec3(1, 1, 1), vec3(1, 1, 0), vec3(1, 0, 0),
    // Face 5: WEST (-X)
    vec3(0, 0, 0), vec3(0, 1, 0), vec3(0, 1, 1), vec3(0, 0, 1)
);

void main() {
    int faceID = gl_VertexID / 6;
    int vertID = gl_VertexID % 6;
```

```
    uint data = faces[faceID];

    int x        = int(data & 0xFu);           // Bits [0-3]
    int y        = int((data >> 4u) & 0xFFu);   // Bits [4-11]
    int z        = int((data >> 12u) & 0xFu);   // Bits [12-15]
    int faceDir  = int((data >> 16u) & 0x7u);   // Bits [16-18]
    int colorIdx = int((data >> 19u) & 0x3Fu);  // Bits [19-24]

    int cornerIdx = QUAD_INDICES[vertID];
    vec3 offset = FACE_OFFSETS[faceDir * 4 + cornerIdx];
    vec3 localPos = vec3(float(x), float(y), float(z)) + offset;
    vec3 worldPos = chunkPos + localPos;

    gl_Position = projection * view * vec4(worldPos, 1.0);
    vColorIndex = colorIdx;
    vFaceDir = faceDir;
    vFragPos = worldPos;
}
```

Côté GPU : Fragment Shader (`chunk2.fs`)

Le fragment shader utilise :

- **Palette r/place 2023** : 33 couleurs (index 0 = air, jamais rendu ; 1–32 = couleurs).
- **Face Brightness** : Luminosité par direction de face pour donner du volume sans vrai calcul d'éclairage.
- **Distance Fog** : Uniforms présents (`fogStart`, `fogEnd`, `fogColor`, `cameraPos`) mais **actuellement désactivés** (commentés).

```
// Face shading — luminosité différente par direction
const float FACE_BRIGHTNESS[6] = float[6](
    1.00,  // 0: TOP     — plein soleil
    0.50,  // 1: BOTTOM  — très sombre
    0.80,  // 2: NORTH   — éclairé
    0.70,  // 3: SOUTH   — un peu sombre
    0.60,  // 4: EAST    — ombre
    0.65   // 5: WEST    — ombre légère
);

void main() {
    vec3 color = PALETTE[vColorIndex];
    color *= FACE_BRIGHTNESS[vFaceDir];
    // fog désactivé pour l'instant
    FragColor = vec4(color, 1.0);
}
```

Vertex Pulling vs MDI : Clarification

Ces deux concepts sont complémentaires mais distincts :

Concept	Rôle
Vertex Pulling	*Comment* le shader lit les données (depuis le SSBO via `gl_VertexID`)
MDI (Multi-Draw Indirect)	*Comment* le CPU dit au GPU de dessiner (1 appel au lieu de N)

Sans MDI — État actuel (`main3.cpp`)

```
CPU (render loop)                          GPU
┌──────────────────────────────┐
│ for (auto& [key, chunk] :    │
│           chunkMap)          │
│   setVec3("chunkPos", ...)   │──→ draw chunk 0
│   chunk->render()            │──→ draw chunk 1
│   ...                        │──→ draw chunk 399
└──────────────────────────────┘

= 400 × glBindBufferBase + 400 × glDrawArrays
= le CPU parle au GPU 800 fois par frame
```

Avec MDI (objectif futur)

```
CPU (render loop)                          GPU
┌──────────────────────────────┐
│ 1 seul SSBO géant avec       │
│ TOUTES les faces             │
│                              │
│ glMultiDrawArraysIndirect()  │──→ draw TOUT en 1 appel
└──────────────────────────────┘

= 1 appel par frame
```

---

3. État Actuel de l'Implémentation

Fichiers Clés

Fichier	Rôle
`include/Chunk2.h`	Classe chunk active : stockage `uint8_t blocks[16][256][16]`, SSBO, Vertex Pulling, face culling inter-chunk
`include/Chunk.h`	Ancienne classe chunk (VBO classique, palette 16 couleurs, ~104 octets/face). **Non utilisée.**
`include/TerrainGenerator.h`	Génération procédurale : 2 couches Simplex Noise (continent + détail) via FastNoiseLite

Fichier	Rôle
`include/LowResRenderer.h`	Pipeline de rendu rétro/PS1 : FBO 640×360 → upscale `GL_NEAREST` → 1920×1080
`include/TextureAtlas.h`	Atlas de textures (non utilisé pour l'instant, palette couleurs uniquement)
`src/main3.cpp`	Point d'entrée : init GLFW/GLAD, boucle de rendu, hashmap de chunks
`src/shader/chunk2.vs`	Vertex Shader : Vertex Pulling + unpacking bits
`src/shader/chunk2.fs`	Fragment Shader : palette 33 couleurs r/place + face shading

Constantes du Chunk (`Chunk2.h`)

```
constexpr uint8_t  CHUNK_SIZE_X = 16;   // Largeur
constexpr uint16_t CHUNK_SIZE_Y = 256;  // Hauteur (0 = bedrock couche)
constexpr uint8_t  CHUNK_SIZE_Z = 16;   // Profondeur
constexpr uint8_t  BEDROCK_LAYER = 0;   // Y = 0 incassable
```

- **Taille mémoire RAM** : `16 × 256 × 16 × 1 octet = 65 536 octets = 64 Ko` par chunk
- **Taille mémoire VRAM** : `faceCount × 4 octets` (1 `uint32_t` par face dans le SSBO)
- Constructeur initialise `blocks` à 0 (air) + couche bedrock (couleur 29, gris foncé) en `y = 0`

Hashmap de Chunks (`main3.cpp`)

Le monde utilise une `unordered_map<int64_t, Chunk2*>` avec une clé calculée par bit shift :

```
// Clé unique : les 32 bits hauts = cx, les 32 bits bas = cz
inline int64_t chunkKey(int cx, int cz) {
    return ((int64_t)cx << 32) | ((int64_t)cz & 0xFFFFFFFF);
}

// Recherche O(1)
inline Chunk2* getChunkAt(int cx, int cz) {
    auto it = chunkMap.find(chunkKey(cx, cz));
    return (it != chunkMap.end()) ? it->second : nullptr;
}
```

> **Note :** C'est plus simple qu'un `ChunkCoord` struct + hash custom (décrit dans la section Architecture). L'implémentation actuelle est un hash implicite par bit shift.

Grille de Chunks

Le code actuel génère une grille fixe de **20×20 = 400 chunks** (`cx` de -10 à 9, `cz` de -10 à 9). La génération se fait en deux passes :

1. **Création + terrain** : Tous les chunks sont créés et remplis via `TerrainGenerator::fillChunk()`.
2. **Mesh** : Les meshes sont générés APRÈS, pour que les voisins existent et que le face culling inter-chunk fonctionne.

Terrain Generator (`TerrainGenerator.h`)

Deux couches de bruit Simplex combinées :

```
Hauteur(x,z) = baseHeight + continent(x,z) × continentAmp + detail(x,z) × detailAmp
```

Paramètre	Valeur	Rôle
`baseHeight`	20	Hauteur de base du sol
`continentAmp`	15.0f	Amplitude grandes collines (±15 blocs)
`detailAmp`	4.0f	Amplitude micro-variations (±4 blocs)
continent freq	0.005	Basse fréquence = formes larges
detail freq	0.02	Haute fréquence = détails fins

Couches de blocs (de bas en haut) :

Y	Bloc	Couleur
0	Bedrock	28 (noir) ou 29 (gris foncé, constructeur)
1 à height-3	Pierre	29 (gris foncé)
height-2 à height-1	Terre	26 (marron)
height	Herbe (surface)	Random 1–32 (sauf 28)

> **Benchmark** : `fillChunkBench()` génère un **damier 3D** (chaque bloc alterne entre solide et air selon `(x+y+z) % 2 == 0`). Utile pour tester le pire cas de face count (toutes les faces visibles).

LowRes Renderer (Pipeline PS1/Rétro)

Le projet utilise un **framebuffer basse résolution** pour un style pixelisé :

```
Scène 3D → Rendu dans FBO 640×360 → Upscale GL_NEAREST → Écran 1920×1080
```

Paramètre	Valeur
`RENDER_WIDTH`	640
`RENDER_HEIGHT`	360
Upscale filter	`GL_NEAREST` (pas d'antialiasing, pixels nets)

Paramètre	Valeur
Écran cible	1920×1080

Le shader d'upscale est un simple quad fullscreen avec `texture(screenTexture, TexCoords)`.

Face Culling Inter-Chunks

`Chunk2::getBlockOrNeighbor()` vérifie les blocs aux bordures en interrogeant le chunk voisin :

```
Chunk West      This Chunk       Chunk East
┌─────────┐    ┌──────────────┐    ┌─────────┐
│         │    │ x=0 ... 15 │    │         │
│  [15]   │←──│  getBlock    │──→│  [0]    │
│         │    │  (-1,y,z)    │    │(16,y,z)│
└─────────┘    └──────────────┘    └─────────┘
```

Si aucun voisin n'existe (`nullptr`) → retourne 0 (air) → la face est créée (le bord du monde est visible).

Distance Fog ✅ Implémenté

Le fog masque la frontière du monde en fondant les blocs distants dans la couleur du ciel. Implémenté dans `chunk2.fs` avec contrôle live via ImGui.

Formule : Fog Linéaire

```
// chunk2.fs — fog linéaire
float dist = length(vFragPos - cameraPos);                        //
distance caméra → fragment
float fogFactor = clamp((dist - fogStart) / (fogEnd - fogStart), 0.0, 1.0);
// 0 = clair, 1 = fog total
color = mix(color, fogColor, fogFactor);                           //
interpolation linéaire
```

```
  fogFactor
  1.0 ──────────────────────────/──────────  ← 100% fog (fogColor)
                                /
                               /
                              /
  0.0 ────────────────────────/              ← 0% fog (couleur du bloc)
       0      fogStart     fogEnd      distance
               80           200        (en unités monde)
```

Comment ça marche :

Distance	fogFactor	Résultat

Distance	fogFactor	Résultat
`dist < fogStart`	0.0	Couleur du bloc pure (pas de fog)
`fogStart < dist < fogEnd`	0.0 – 1.0	Mélange progressif bloc → fog
`dist > fogEnd`	1.0	Couleur du fog pure (bloc invisible)

`mix(a, b, t)` = interpolation linéaire : `a × (1-t) + b × t`

Pourquoi fogColor = glClearColor ?

```
Si fogColor ≠ glClearColor :
┌──────────────────────────────────────┐
│   terrain   |  fog gris  | ciel bleu|  ← transition VISIBLE = moche
└──────────────────────────────────────┘

Si fogColor = glClearColor :
┌──────────────────────────────────────┐
│   terrain  ──  fondu ─── ciel        |  ← seamless !
└──────────────────────────────────────┘
```

La `glClearColor` dans `main3.cpp` est automatiquement synchronisée avec `FOG_COLOR` :

```
glClearColor(FOG_COLOR.r, FOG_COLOR.g, FOG_COLOR.b, 1.0f);
```

Valeurs actuelles

```
// main3.cpp — modifiables via ImGui sliders
float fogStart = 80.0f;   // Le fog commence à 80 blocs
float fogEnd   = 200.0f;  // À 200 blocs, c'est 100% fog
```

> ✅ **Astuce :** Le fog va bien avec le frustum culling. Si `fogEnd < far plane`, les chunks au-delà du fog sont de toute façon invisibles → le frustum culling les skip. Double économie.

---

4. Optimisations GPU (Pipeline de Rendu)

Face Culling Avancé

- **Blocs standards :** Ne jamais rendre les faces partagées entre deux blocs opaques adjacents. ✅ *Implémenté dans `meshGenerate()`.*
- **Blocs Custom (Blockbench) :** Si une face touche les limites de la bounding box et qu'un bloc plein est collé contre, la face peut être cullée. *(Futur)*

- **Bordures de Chunk :** Si le chunk voisin est `nullptr`, la face est créée (air). ✅ *Implémenté via `getBlockOrNeighbor()`.*

Z-Prepass (Depth Prepass) — ❌ PAS rentable pour VoxPlace

Le Z-Prepass réduit l'**Overdraw** (calculs inutiles de pixels cachés) à zéro. Le Fragment Shader ne s'exécute que pour les pixels réellement visibles.

Principe :

```
Passe 1 : Profondeur seule (pas de couleur)
┌──────────────────────────────────┐
│ glColorMask(FALSE, FALSE, ...)   │  ← Fragment Shader = OFF
│ Vertex Shader → Depth Buffer     │  ← On remplit juste le Z-buffer
└──────────────────────────────────┘

                  ↓
Passe 2 : Couleur (depth = EQUAL)
┌──────────────────────────────────┐
│ glColorMask(TRUE, TRUE, ...)     │
│ glDepthFunc(GL_EQUAL)            │  ← Seuls les pixels "gagnants"
│ Fragment Shader = ON             │     exécutent le FS
└──────────────────────────────────┘
```

Pourquoi c'est une PERTE pour VoxPlace

Le Z-Prepass est un **trade-off** : on exécute le Vertex Shader **2×** pour exécuter le Fragment Shader **1×** au lieu de ~1.5×. C'est rentable **seulement** si le Fragment Shader est cher.

```
COÛT DU FRAGMENT SHADER DE VOXPLACE (chunk2.fs) :

vec3 color = PALETTE[vColorIndex];        // 1 lookup tableau
color *= FACE_BRIGHTNESS[vFaceDir];       // 1 multiplication
FragColor = vec4(color, 1.0);             // 1 write

Total : ~3 opérations GPU. C'est RIEN.
```

```
RÉSOLUTION DE RENDU :

640 × 360 = 230 400 pixels       ← VoxPlace (LowRes PS1)
1920 × 1080 = 2 073 600 pixels   ← Full HD

Le fragment shader s'exécute sur 9× MOINS de pixels qu'en full HD.
Même avec de l'overdraw, le coût total est négligeable.
```

	Sans Z-Prepass	Avec Z-Prepass

	Sans Z-Prepass	Avec Z-Prepass
Vertex Shader	1× tous les vertices	2× tous les vertices
Fragment Shader	~1.3× les pixels (overdraw)	1× les pixels
Bilan VoxPlace	✅ VS cheap + FS cheap	❌ 2× VS pour économiser un FS déjà gratuit

Quand le Z-Prepass SERAIT rentable

Situation	Rentable ?
Fragment Shader complexe (PBR, 10 lights, shadow maps)	✅ OUI
Rendu full HD ou 4K	✅ OUI
Beaucoup de couches superposées (transparence)	✅ OUI
VoxPlace (3 opérations FS, 640×360, opaque)	❌ NON

> ⚠ **Conclusion (mise à jour) :** Avec le FS actuel (3 opérations, 640×360), le Z-Prepass est une perte nette. **MAIS** si VoxPlace ajoute :
>
> - **Ambient Occlusion** → FS passe de 3 à ~10+ opérations
> - **Résolution dynamique** → potentiellement 1920×1080 (9× plus de pixels)
>
> Alors le Z-Prepass redevient **rentable** et devrait être implémenté **après** l'AO, avec un toggle (F3) pour comparer.

Frustum Culling — ✅ PRIORITÉ #1

Le frustum culling est l'optimisation la plus impactante **maintenant** car elle élimine des chunks **entiers** du pipeline GPU. C'est un test **CPU** : le GPU ne fait rien de plus.

C'est quoi le Frustum ?

Le "frustum" c'est la **pyramide tronquée** de la caméra — tout ce que la caméra voit. Tout objet en dehors de cette pyramide est invisible et ne devrait pas être envoyé au GPU.

```
Vue de dessus (la caméra regarde vers le haut du schéma) :

            near plane
             ┌─────┐
            /       \
           /  VISIBLE \
          /   zone du   \
         /    frustum     \
        /                 \
       /                   \
      └─────────────────────┘
            far plane
```

```
        🎥 caméra (ici)

    Vue de côté :

         near         far
          |            |
          |   /────────|  ← top plane
          |  /         |
          | 🎥         |  ← centre
          |  \         |
          |   \────────|  ← bottom plane
          |            |
```

Le frustum est défini par **6 plans** : Left, Right, Top, Bottom, Near, Far. Chaque plan "coupe" l'espace en deux : le côté visible et le côté invisible.

Comment extraire les 6 plans ?

On les extrait directement de la matrice **VP** (projection × view). C'est la méthode de Gribb & Hartmann (1999) — chaque plan est une combinaison de lignes de la matrice :

```
    VP = Projection × View   (matrice 4×4)

    Chaque ligne de VP :
    row[0] = VP[0][0], VP[1][0], VP[2][0], VP[3][0]
    row[1] = VP[0][1], VP[1][1], VP[2][1], VP[3][1]
    row[2] = VP[0][2], VP[1][2], VP[2][2], VP[3][2]
    row[3] = VP[0][3], VP[1][3], VP[2][3], VP[3][3]

    Les 6 plans :
    Left   = row[3] + row[0]     "tout ce qui est trop à gauche"
    Right  = row[3] - row[0]     "tout ce qui est trop à droite"
    Bottom = row[3] + row[1]     "tout ce qui est trop en bas"
    Top    = row[3] - row[1]     "tout ce qui est trop en haut"
    Near   = row[3] + row[2]     "tout ce qui est trop près"
    Far    = row[3] - row[2]     "tout ce qui est trop loin"
```

Chaque plan a la forme `ax + by + cz + d = 0` (équation du plan dans l'espace 3D). Les coefficients `(a, b, c)` forment la **normale** du plan, et `d` est la distance à l'origine.

Comment tester un chunk vs le frustum ?

Chaque chunk a une **AABB** (Axis-Aligned Bounding Box) :

```
    Chunk (cx=3, cz=5) :

    min = (3×16, 0, 5×16) = (48, 0, 80)
    max = (48+16, 256, 80+16) = (64, 256, 96)
```

```
        max (64, 256, 96)
        /──────────/|
       /          / |    256 blocs de haut
      /──────────/  |
      |          |  |
      |   CHUNK  |  /
      |          | /     16×16 au sol
      └──────────/
    min (48, 0, 80)
```

Pour chaque plan du frustum, on teste le **point le plus éloigné de l'AABB dans la direction de la normale** (le "positive vertex"). Si ce point est du mauvais côté du plan → **le chunk est invisible**.

```
Test : AABB vs 1 plan

Plan avec normale n = (nx, ny, nz)

On prend le coin de l'AABB le plus aligné avec n :

pVertex.x = (nx >= 0) ? max.x : min.x
pVertex.y = (ny >= 0) ? max.y : min.y
pVertex.z = (nz >= 0) ? max.z : min.z

distance = dot(n, pVertex) + d

Si distance < 0 → le chunk est ENTIÈREMENT de l'autre côté du plan
                 → INVISIBLE, on skip le rendu ✅
```

On répète ce test pour les 6 plans. Si le chunk est du bon côté des 6 plans, il est visible.

Algorithme complet

```
Pour chaque frame :
  1. Calculer VP = projection × view
  2. Extraire les 6 plans (6 vec4)
  3. Normaliser chaque plan (diviser par length(normal))

Pour chaque chunk :
  1. Calculer min/max de l'AABB
  2. Pour chaque plan (6 tests) :
     - Trouver le positive vertex
     - Calculer distance signée
     - Si distance < 0 → SKIP ce chunk
  3. Si aucun plan ne l'exclut → RENDER

Complexité : 6 dot products par chunk = ~24 multiplications
C'est RIEN pour le CPU. Même pour 10 000 chunks.
```

> ✅ **Le frustum culling est gratuit côté GPU** (c'est un test CPU), réduit tout le pipeline de ~3×, et n'a aucun trade-off. C'est TOUJOURS la première optimisation à implémenter. ✅ **Implémenté dans `include/Frustum.h`** — méthode Gribb-Hartmann.

Ambient Occlusion Per-Vertex ✅ Implémenté

L'AO voxel assombrit les coins et arêtes entre blocs pour donner du volume. C'est du **per-vertex AO** calculé au moment du meshing (CPU), pas du SSAO screen-space (GPU).

Principe : 3 blocs voisins par vertex

Pour chaque vertex d'une face, on examine 3 blocs voisins dans la direction de la face :

```
  Vue d'en haut d'un vertex sur une face TOP (+Y) :

  Le vertex V est au coin. On regarde les 3 blocs autour de lui
  dans la couche y+1 (au-dessus du bloc courant).

      side1    corner
      ┌───┐ ┌───┐
      | ? || ? |    Si side1 ET side2 sont solides → AO = 0 (max ombre)
      └───┘ └───┘    Sinon : AO = 3 - (side1 + side2 + corner)
  ┌───┐
  | V | ← vertex        AO = 3 : lumineux (0 voisins solides)
  └───┘                  AO = 2 : léger ombrage (1 voisin solide)
      ┌───┐              AO = 1 : ombré (2 voisins solides)
      | ? | side2         AO = 0 : très sombre (3 voisins solides, ou
s1+s2)
      └───┘
```

Formule AO

```
// Chunk2.h — computeVertexAO()
int computeVertexAO(side1, side2, corner) {
    if (side1 && side2) return 0;  // Cas spécial : coin totalement
occluded
    return 3 - (side1 + side2 + corner);
}
```

Le cas spécial `side1 && side2` est important : si les deux côtés sont solides, le corner ne peut pas être visible même s'il est vide → on force AO = 0.

```
  AO = 3        AO = 2        AO = 1        AO = 0
  ┌───┐         ┌───┐         ┌▓▓▓┐         ┌▓▓▓┐
  |   | air     |   | air     |▓▓▓| solide  |▓▓▓| solide
  └───┘         └───┘         └───┘         └───┘

  ┌───┐         ┌───┐         ┌───┐         ┌───┐
```

```
    │ V │          │ V │          │ V │          │ V │
    └───┘          └───┘          └───┘          └───┘

    ┌───┐          ┌───┐          ┌───┐          ┌───┐
    │   │ air      │ ■ │ solide   │   │ air      │ ■ │ solide
    └───┘          └───┘          └───┘          └───┘

    0 voisins      1 voisin       1+corner       2 côtés (forcé)
```

Table de 72 offsets

Pour chaque face (6) × chaque vertex (4) × chaque voisin (3 = side1, side2, corner), on définit un offset (dx, dy, dz) relatif au bloc. Total : 6×4×3 = 72 vecteurs, définis dans `AO_OFFSETS[6][4][3][3]` dans `Chunk2.h`.

```
    Exemple : Face TOP (+Y), vertex v0 (coin 0,1,0) du bloc (x,y,z)

    Couche au-dessus (y+1) :
           z-1      z       z+1
    x-1  [corner] [side1] [  ]       side1  = getBlock(x-1, y+1, z)
    x    [side2]  [FACE]  [  ]       side2  = getBlock(x, y+1, z-1)
    x+1  [  ]     [  ]    [  ]       corner = getBlock(x-1, y+1, z-1)
```

Nouveau Bit Layout (32/32 bits utilisés)

```
    Ancien (sans AO) :
    [XXXX][YYYYYYYY][ZZZZ][FFF][CCCCCC][???????]
     4      8        4     3    6 bits   7 libres = 25/32

    Nouveau (avec AO) :
    [XXXX][YYYYYYYY][ZZZZ][FFF][CCCCC][AA][AA][AA][AA]
     4      8        4     3    5 bits  v0  v1  v2  v3  = 32/32

    Changement :
    - Color réduit de 6 → 5 bits (color-1, car 0=air jamais packé)
    - Les 8 bits libérés stockent 4 valeurs AO × 2 bits = 0-3 chacune
```

Champ	Bits	Plage	Remarque
X	[0-3]	0-15	Position locale
Y	[4-11]	0-255	Position locale
Z	[12-15]	0-15	Position locale
Face	[16-18]	0-5	Direction
Color	[19-23]	0-31	**Packé comme color-1** (shader fait +1)

Champ	Bits	Plage	Remarque
AO v0	[24-25]	0-3	Vertex 0
AO v1	[26-27]	0-3	Vertex 1
AO v2	[28-29]	0-3	Vertex 2
AO v3	[30-31]	0-3	Vertex 3

Pipeline GPU : Vertex → Fragment

```
Vertex Shader (chunk2.vs)
┌──────────────────────────────────────────────────┐
│ 1. Unpack AO pour les 4 vertices de la face      │
│ 2. cornerIdx = QUAD_INDICES[vertID] → 0,1,2,3    │
│ 3. ao = AO_CURVE[aoValues[cornerIdx]]            │
│    AO_CURVE = [0.20, 0.50, 0.75, 1.00]           │
│ 4. vAO = ao (PAS flat → interpolé par rasterizer)│
└──────────────────────────────────────────────────┘
                        ↓
Rasterizer (GPU hardware)
┌──────────────────────────────────────────────────┐
│ Interpole vAO entre les 3 sommets du triangle    │
│ → gradient doux sur toute la face ! 🎨           │
└──────────────────────────────────────────────────┘
                        ↓
Fragment Shader (chunk2.fs)
┌──────────────────────────────────────────────────┐
│ color *= vAO;  ← simple multiplication           │
└──────────────────────────────────────────────────┘
```

> ✅ **Point clé :** `vAO` n'est **PAS** `flat` (contrairement à `vColorIndex` et `vFaceDir`). Le rasterizer GPU interpole automatiquement la valeur entre les 3 sommets de chaque triangle, créant un dégradé doux d'ombre vers lumière sur la face. C'est **gratuit** — le GPU le fait de toute façon.

Résumé des optimisations par priorité

```
Priorité     Optimisation            Gain              Quand ?
──────────────────────────────────────────────────────────────
☆ 1         Frustum Culling         ~3× draw calls    Maintenant
2            Fog                     Masque bordures    Maintenant
3            Ambient Occlusion       Visuel ++          Prochain
4            Résolution dynamique    Toggle PS1/HD      Prochain
5            Z-Prepass               ~1.5× FS (si AO)  Après AO
...          MDI, Multithreading     Futur              Plus tard
```

---

5. Optimisations CPU (Types, Cache & Mémoire)

> Piège classique : confondre optimisation du **stockage** (RAM/Cache) et optimisation du **calcul** (CPU/Registres).

Pourquoi `int` pour les boucles ?

Les processeurs modernes manipulent nativement des mots de **32 ou 64 bits**. C'est la taille de leurs registres.

- **Performance :** Utiliser `uint8_t` dans un `for` force le CPU à effectuer des opérations de masquage supplémentaires à chaque itération pour simuler un comportement 8 bits.
- **Risque d'Overflow :** `for (uint8_t y = 0; y < 256; y++)` → **boucle infinie** ! `y` passe de 255 à 0 (overflow), donc `y < 256` est toujours vrai.

→ **Toujours utiliser `int` pour les compteurs de boucle.** C'est la taille naturelle du registre CPU : rapide et sans risque d'overflow.

> ✅ **VoxPlace le fait correctement** : toutes les boucles dans `Chunk2.h`, `TerrainGenerator.h` utilisent `int` pour les itérateurs `x`, `y`, `z`.

Pourquoi `uint8_t` pour le stockage ?

Le **cache CPU** (L1, L2) est ultra-rapide mais très petit. Plus les données sont compactes, plus on en charge d'un coup.

Type de bloc	Taille d'un chunk `[16][256][16]`	Tient en cache ?
`uint8_t` (1 octet)	**64 Ko**	✅ Cache L1 ou L2
`int` (4 octets)	**256 Ko**	❌ Allers-retours RAM

> ✅ **VoxPlace** : `uint8_t blocks[CHUNK_SIZE_X][CHUNK_SIZE_Y][CHUNK_SIZE_Z]` → 64 Ko pile.

Gestion de la hauteur maximale (Y = 255)

Option	Description
A — Toit de verre	Construire jusqu'à 255, ignorer au-dessus. Le moteur physique tolère le joueur "hors tableau".
B — Faux plafond	Bloquer à 254 (`if (y >= 254) return false;`), air garanti au-dessus.

> État actuel : `setBlock()` vérifie `y < CHUNK_SIZE_Y` (256), donc on peut placer un bloc en `y = 255`. `getBlock()` retourne 0 (air) pour `y >= 256`. Approche **A** de facto.

Résumé : Registres vs Cache

Contexte	Type idéal	Raison
Calcul / Boucles (Registres CPU)	`int` / `int64_t`	Taille native du registre, pas de masquage

Contexte	Type idéal	Raison
Stockage / Données (Cache CPU)	`uint8_t`	Maximise la densité en cache, réduit les accès RAM

> **Le pattern idéal : Stockage compact (`uint8_t`) + Calcul large (`int`).**

---

6. Multithreading (CPU)

> Source : LowLevelGameDev — Multithreading Voxel Engine

La pire erreur est de mettre des `mutex` (verrous) partout entre le thread principal et les workers. Cela force les threads à s'attendre, annulant le gain de performance.

Modèle "Thread Pool" Sans Mutex (Data-Oriented Design)

L'utilisation excessive de mutex **détruit** les performances. L'approche correcte est un Thread Pool avec des **points de synchronisation stricts** et **zéro communication pendant l'exécution** :

```
Main Thread                    Thread Pool                    GPU
    |                              |                            |
    |  1. Emballe TOUTES les       |                            |
    |     données nécessaires      |                            |
    |──────────────────────────────→|                            |
    |                              |  2. Calcul en isolation    |
    |     ✗ Aucune communication   |     (Perlin, Meshing...)   |
    |     pendant cette phase      |     ✗ Ne demande RIEN au   |
    |                              |        main thread          |
    |                              |                            |
    |  3. Point de synchronisation |                            |
    |     Récupère les buffers    ←|                            |
    |                              |                            |
    |  4. Upload OpenGL ──────────────────────────────────────→|
    |     (main thread obligatoire)|                            |
```

Règles :

1. Le **Main Thread** prépare un ensemble de tâches **indépendantes** (ex: "génère le mesh du chunk (3,5) avec ces données de voisins").
2. Il fournit **toutes** les données nécessaires d'avance — pas d'accès partagé.
3. Il envoie les tâches au **Thread Pool**.
4. Pendant l'exécution : **aucune communication** entre les threads. Zéro mutex.
5. Le Main Thread attend à un **point de synchronisation fixe**, récolte les résultats.
6. Il upload au GPU (OpenGL exige le main thread).

> ⚠ **État actuel :** VoxPlace ne fait **pas** de multithreading. Tout (génération terrain, meshing, upload) se fait sur le main thread, de façon synchrone au démarrage.

---

7. Architecture Serveur & Réseau

Le serveur doit être pensé comme un OS gérant des ressources limitées : **budget de 50ms** par boucle pour tenir les **20 Ticks Par Seconde (TPS)**.

Time Slicing & Throttling

Ressource	Limite	Raison
Génération de chunks	1 chunk / tick	Évite de freeze la logique physique/réseau
Envoi réseau	5 chunks / joueur / tick	Évite la saturation des buffers TCP/UDP (→ timeout)

Multithreading Spatial (Régions)

- Regrouper les chunks en zones isolées ("Régions").
- Si Joueur A et Joueur B sont très éloignés (régions disjointes), on peut assigner chaque région à un thread séparé → physique et réseau en parallèle, **zéro mutex**.

Simulation Distance vs Render Distance

Paramètre	Géré par	Exemple	Rôle
Render Distance	Client	64 chunks	Affichage visuel uniquement
Simulation Distance	Serveur	6 chunks	Physique, Random Tick Updates (pousse, fluides)

> Ne **jamais** appliquer les Random Tick Updates sur toute la Render Distance. Les chunks éloignés dorment en RAM.

---

8. Architecture VoxPlace : Lazy Generation

Concept Core : Moteur Client-Serveur où la génération procédurale (Perlin Noise) est pilotée dynamiquement par l'exploration et le placement de blocs par les joueurs. Pas de structures prédéfinies.

Spatial Hashing (Table de Hachage)

Le monde n'alloue de la mémoire **QUE** pour les chunks qui existent. Recherche en `O(1)`.

Méthode recommandée (avec struct custom) :

```
struct ChunkCoord {
    int x, z;
    bool operator==(const ChunkCoord& o) const {
        return x == o.x && z == o.z;
    }
};

struct ChunkHash {
    std::size_t operator()(const ChunkCoord& pos) const {
        return std::hash<int>()(pos.x) ^ (std::hash<int>()(pos.z) << 1);
```

```
    }
};

std::unordered_map<ChunkCoord, Chunk2*, ChunkHash> activeChunks;
```

Méthode actuelle (implémentation simplifiée dans `main3.cpp`) :

```
// Clé int64_t par bit shift — plus simple, pas besoin de struct/hash
std::unordered_map<int64_t, Chunk2*> chunkMap;

inline int64_t chunkKey(int cx, int cz) {
    return ((int64_t)cx << 32) | ((int64_t)cz & 0xFFFFFFFF);
}
```

Trigger de Génération : `setBlock`

- Le déclencheur n'est **plus** le déplacement, mais la **modification du monde**.
- Si un joueur pose un bloc en limite de chunk (ex: `x == 15`), le serveur vérifie la `unordered_map`. Si le chunk voisin n'existe pas, il est instancié **asynchroneusement** via le Perlin Noise.
- **Gestion du Vide :** Les chunks non générés sont traités comme des **murs invisibles** (Solid Colliders) pour empêcher les joueurs de tomber dans le vide.

> ⚠ **État actuel :** Pas encore de lazy generation. Tous les 400 chunks sont pré-générés au démarrage de façon synchrone.

---

9. Rendu Dynamique & UX des Bordures

L'apparition soudaine d'un chunk provoque un **stutter** (chute brutale de FPS).

Asynchronous Upload & Time-Slicing (Anti-Stutter)

- **Problème :** `glBufferData` d'un mesh entier en une frame bloque le CPU/GPU.
- **Solution :** File d'attente → upload via `glBufferSubData` par petites tranches (ex: 10% du mesh par frame) ou via `glMapBufferRange`.

Animations d'Apparition (Vertex Shader)

- Passer un Uniform `float u_spawnTime` au shader lors de la création du chunk.
- Si le temps actuel est proche de `u_spawnTime`, modifier la position Y ou l'échelle → les blocs semblent **"pousser" du sol**.

Distance Fog

- Brouillard exponentiel (`EXP2`) ou linéaire dans le Fragment Shader pour masquer la frontière du monde.
- La couleur du brouillard doit correspondre à la `glClearColor` (Skybox) pour un fondu parfait.

- ✅ **Implémenté** dans `chunk2.fs` (uniforms `fogStart`, `fogEnd`, `fogColor`) mais **désactivé** (`FOG_START = FOG_END = 0`).

---

10. Les "Fausses" Bonnes Idées (À Éviter)

❌ Greedy Meshing

Fusionner les faces adjacentes identiques en un grand quad semble optimal, mais :

- **Coût CPU** : Algorithme lourd à recalculer à chaque modification de bloc.
- **Incompatible AO** : Chaque sous-section du quad fusionné aurait besoin d'une ombre différente.
- **Gain négligeable** : Avec le Vertex Pulling, les sommets pèsent 4 octets. Le GPU préfère beaucoup de petits triangles simples.
- **Alternative** : Simple **Face Culling** (ne pas dessiner les faces cachées). Ultra-rapide et suffisant. ✅ *C'est ce que fait VoxPlace.*

❌ Génération côté Client

Demander au client de générer les chunks à partir du Seed :

- Bugs de **désynchronisation** massifs (structures qui se chevauchent).
- Un chunk compressé (quelques Ko) se transfère en < 1 seconde via le réseau.

---

11. To-Do

- [x] **Cross-Chunk Face Culling** — `getBlockOrNeighbor()` + `unordered_map` ✅
- [x] **Terrain Perlin Noise** — FastNoiseLite, 2 couches Simplex ✅
- [x] **ImGui** — Debug UI (FPS, chunks, faces, vertices, camera, speed slider) ✅
- [x] **Frustum Culling** — `Frustum.h`, test AABB vs 6 plans, compteur ImGui ✅
- [x] **Fog** — Fog linéaire décommenté, sliders ImGui (fogStart/fogEnd) ✅
- [x] **Ambient Occlusion** — Per-vertex AO, 4×2 bits packés, AO_CURVE interpolée ✅
- [x] ~~**Z-Prepass**~~ — ❌ Pas rentable actuellement. À réévaluer après AO + full res.
- [ ] **Front-to-back Sorting** — Trier les chunks par distance à la caméra (réduit overdraw)
- [ ] **Résolution dynamique** — Toggle PS1 640×360 ↔ Full HD 1920×1080 (F11)
- [ ] **Indirect Rendering** — `glMultiDrawArraysIndirect` : 1 seul draw call pour tous les chunks
- [ ] **Chunk Sections** — Diviser le chunk en sous-sections de 16^3 (skip sections vides)
- [ ] **LODs** — Level of Detail : fusionner les blocs éloignés (2×2 puis 4×4) pour simplifier la géométrie
- [ ] **Génération serveur** — Lazy generation côté serveur
- [ ] **Multithreading** — Thread pool sans mutex pour `meshGenerate()` et `fillChunk()`
- [ ] **Dirty flag rebuild** — Max 1 chunk rebuild par frame
- [ ] **Load/unload dynamique** — Chunks chargés par render distance autour de la caméra
- [ ] **Day/Night Cycle** — Interpolation lumière + skybox dynamique

---

11b. Analyse de ourCraft (LowLevelGameDev)

> Référence : github.com/meemknight/ourCraft — moteur voxel très avancé avec PBR, shadows, SSAO, multiplayer.

Optimisations de ourCraft pertinentes pour VoxPlace

1. Front-to-Back Chunk Sorting (✅ facile à implémenter)

ourCraft trie les chunks **par distance à la caméra** avant de les dessiner (les plus proches d'abord). Cela exploite le **Early-Z rejection** du GPU :

```
Sans tri (ordre aléatoire)              Avec tri (front-to-back)
┌──────────────────────────────┐        ┌──────────────────────────────┐
│ chunk loin → écrit depth     │        │ chunk proche → écrit depth   │
│ chunk proche → écrit depth   │        │ chunk loin → SKIP (depth     │
│   mais le loin reste !       │        │   test fail = pas de FS) !   │
└──────────────────────────────┘        └──────────────────────────────┘

FS exécuté pour des pixels qui sont ensuite cachés  →  OVERDRAW
Le GPU dessine le bloc proche SUR le bloc loin → gaspillage

Avec front-to-back, le depth buffer est rempli d'abord par les objets
proches, et les objets loin échouent au depth test → le GPU skip
automatiquement le fragment shader = Z-cull hardware
```

Implémentation (dans le render loop) :

```
// Trier les chunks par distance² au joueur (pas besoin de sqrt)
std::sort(chunkVector.begin(), chunkVector.end(),
    [camX, camZ](Chunk* a, Chunk* b) {
        int ax = a->chunkX - camX, az = a->chunkZ - camZ;
        int bx = b->chunkX - camX, bz = b->chunkZ - camZ;
        return (ax*ax + az*az) < (bx*bx + bz*bz);  // Plus proche en
premier
    });
```

> ✅ **C'est un Z-Prepass "gratuit"** grâce au hardware Early-Z du GPU. Pas de double pass, pas de changement de shader. Juste un `std::sort` une fois par frame.

2. Chunk Sections (subdiviser en 16³)

ourCraft ne génère pas le mesh pour les Y-levels vides. Actuellement VoxPlace a un chunk de 16×256×16 — la boucle itère les 65 536 blocs même si la plupart sont de l'air.

```
Chunk actuel (16 × 256 × 16)      Avec sections (16 × 16 × 16 × 16)
┌──────────────────────┐          ┌──────────────────────┐
│ AIR AIR AIR AIR AIR  │  y=256   │ section 15 (SKIP!)   │
│ AIR AIR AIR AIR AIR  │          │ section 14 (SKIP!)   │
│ AIR AIR AIR AIR AIR  │          │ section 13 (SKIP!)   │
│ ...                  │          │ ...                  │
│ AIR AIR AIR AIR AIR  │          │ section  3 (SKIP!)   │
```

```
    | TERRAIN                |  y=35    | section  2 (MESH!)    |
    | STONE STONE STONE      |  y=16    | section  1 (MESH!)    |
    | BEDROCK                |  y=0     | section  0 (MESH!)    |
    └────────────────────────┘          └───────────────────────┘

    Itère 65 536 blocs                   Itère seulement 3×4096 = 12 288 blocs
    (93% sont de l'air → gaspillage)     (skip les 13 sections vides)
```

3. Indirect Rendering (glMultiDrawArraysIndirect)

> **Question du To-Do : "Indirect rendering est une bonne idée ?"**

Réponse : Oui, mais **pas maintenant**. Indirect rendering remplace 400 `glDrawArrays` par 1 `glMultiDrawArraysIndirect`. C'est pertinent quand :

- Le nombre de chunks visibles dépasse ~2000
- Le CPU est le bottleneck (pas le GPU)
- Le monde est relativement statique

Pour VoxPlace **avec les modifications de blocs en temps réel** (multijoueur), MDI ajoute de la complexité (buffer géant + fragmentation). L'approche actuelle (1 SSBO par chunk) est meilleure pour les mises à jour dynamiques.

Quand l'implémenter : Après le multithreading et le load/unload dynamique, quand on aura beaucoup plus de chunks.

5. Level of Detail (LODs) — Géométrie simplifiée à distance

> Source : LowLevelGameDev — Chunk system & LODs

Pour afficher de **très grandes distances** (64+ chunks), le GPU ne peut pas traiter la même densité de triangles partout. Les LODs fusionnent les blocs éloignés pour réduire la géométrie :

```
    Distance       LOD    Résolution    Blocs
    ──────────────────────────────────────────
    0-16 chunks    LOD0   1×1 (normal)  16×256×16
    16-32 chunks   LOD1   2×2 (merge)   8×128×8      ← 4× moins de faces
    32-64 chunks   LOD2   4×4 (merge)   4×64×4       ← 16× moins de faces

    Vue de dessus :

    LOD0 (proche)     LOD1 (moyen)     LOD2 (loin)
    ┌─┬─┬─┬─┐         ┌───┬───┐          ┌─────┐
    ├─┼─┼─┼─┤         │   │   │          │     │
    ├─┼─┼─┼─┤    →    ├───┼───┤     →    │     │
    ├─┼─┼─┼─┤         │   │   │          │     │
    └─┴─┴─┴─┘         └───┴───┘          └─────┘
    16 faces            4 faces            1 face
```

Le challenge : le CPU/serveur doit pouvoir suivre la charge de génération des chunks LOD. C'est pas juste du rendering — il faut aussi des versions simplifiées des données de blocs.

Pour VoxPlace : Pas prioritaire tant qu'on n'a pas le load/unload dynamique et le multithreading. La render distance actuelle (20 chunks) ne nécessite pas de LODs.

6. Architecture technique de ourCraft (deep-dive du code source)

> Source locale : `/home/alpha/Documents/TFE/ourCraft/`

Vertex Packing — 3 ints par vertex (SSBO)

ourCraft utilise **3 entiers** par vertex envoyés via vertex attributes, avec le geometry lookup dans un **SSBO** :

```
// rendering/chunk.cpp — pushFlagsLightAndPosition()
// Format du int Y :
//   0x    FF      FF      FF    FF
//      —flags——light——position—
//  flags: isWater(1 bit) + isInWater(1 bit) + aoShape(4 bits haut)
//  light: sunLight(4 bits) + torchLight(4 bits) mergés
//  position: y (16 bits)

// pushFaceShapeTextureAndColor()
// Format du int texture :
//   0bxxxxx000'00000000
//    color(5b)  texture(11b)  → 2048 textures max + 32 couleurs

// Layout vertex = 3 ints :
// [0] x position (int)
// [1] y position | light | flags (packed)
// [2] z position (int)
```

Comparaison avec VoxPlace :

	ourCraft	VoxPlace
Données/vertex	3 ints (12 bytes)	1 uint32 (4 bytes)
Geometry	SSBO lookup → `vertexData[orientation * 12 + vertexId * 3]`	Reconstruit dans VS
Textures	2048 via bindless (albedo + normal + material + parallax)	32 couleurs palette
Lumière	Sun (4 bits) + Torch (4 bits) par face	Aucune (face shading fixe)
AO	aoShape (4 bits dans flags)	4×2 bits dans packed data

Face Brightness — Facteurs extraits du vertex shader

```
// defaultShader.vert — ligne 26-62
float vertexColor[] = float[](
    0.95,  // front
    0.85,  // back
    1.0,   // top
    0.8,   // bottom
    0.85,  // left
    0.95   // right
    // + grass variants, leaves variants...
);

// Appliqué avec la lumière :
v_ambient = vertexColor[faceOrientation] * (ambientInt / 15.0);
```

Comparaison face shading :

Face	ourCraft	BetterSpades	VoxPlace
TOP	1.00	1.00	1.00
BOTTOM	0.80	0.50	0.50
FRONT	0.95	0.875	0.80
BACK	0.85	0.625	0.70
LEFT	0.85	0.75	0.65
RIGHT	0.95	0.75	0.60

> ourCraft a des facteurs plus proches (0.80-1.0) → contraste plus doux car il compte sur l'éclairage dynamique pour la profondeur.

LOD — Implémentation réelle dans le code

```
// chunkSystem.cpp — determineLodLevel()
int determineLodLevel(playerChunkPos, chunkPos) {
    float distPercent[6] = {0, 0.8, 0.7, 0.6, 0.5, 0.5};
    int distMax[6]       = {99, 30, 24, 16, 12, 8};

    int firstLod = distPercent[lodStrength] * viewDistance;
    firstLod = min(firstLod, distMax[lodStrength]);

    if (distSquared > firstLod²) return 1;  // LOD1 = bake simplifié
    else return 0;                           // LOD0 = full detail
}
```

Le LOD est un argument passé directement à `bakeAndDontSendDataToOpenGl()` qui change la résolution du meshing. Binaire (LOD0/LOD1), pas de niveaux intermédiaires.

Animations vertex shader — Eau et herbe

```
// Herbe : déplacement sinusoïdal basé sur position monde + temps
float offset = cos((facePosition.x * dir.x +
    facePosition.z * dir.y - u_timeGrass * SPEED) * FREQUENCY) * AMPLITUDE;
vertexShape.x += grassMask[vertexId] * offset;

// Eau : double couche d'ondes (rapide + lente)
vertexShape.y += waterMask * (offset_rapide + offset_lente - 0.08);
```

> Pour VoxPlace (filtre PS1), une animation d'eau en vertex shader serait très stylée et peu coûteuse.

Multithreading — ThreadPool avec données per-thread

```
// chunkSystem.cpp
struct PerThreadData {
    vector<TransparentCandidate> transparentCandidates;
    vector<int> opaqueGeometry;
    vector<int> transparentGeometry;
    vector<ivec4> lights;
    bool updateTransparency, updateGeometry;
    Chunk* chunk;
};

// Chaque thread a SON propre set de buffers → zéro contention
void bakeWorkerThread(int index, ThreadPool& threadPool) {
    auto& data = perThreadData[index];
    bakeLogicForOneThread(threadPool, data.opaque, data.transparent, ...);
    threadPool.threIsWork[index] = false;  // Signal terminé
}
```

> Confirme l'approche "zero mutex" — chaque worker a ses propres buffers, le main thread collecte les résultats et fait l'upload GPU (OpenGL = single thread only).

Pipeline post-processing complet (43 shaders !)

Shader	Fichier	Pertinence VoxPlace
Z-Prepass	`rendering/zpass.vert/frag`	Pas besoin (front-to-back suffit)
HBAO (SSAO avancé)	`postprocess/hbao.frag`	✗ On a du per-vertex AO
Tone Mapping	`postprocess/toneMap.frag`	Possible futur
Bloom (multi-pass)	`filterDown + filterBloom + applyBloom`	Esthétique

Shader	Fichier	Pertinence VoxPlace
FXAA	`postprocess/fxaa.frag`	❌ Pixels nets voulus (PS1)
SSR	`postprocess/ssr.frag`	❌ Pas d'eau prévue
Atmospheric Scattering	`skybox/atmosphericScattering.frag`	✅ Day/night cycle
Shadow Maps	`rendering/renderDepth.vert/frag`	Futur lointain
Radial Blur	`postprocess/radialBlur.frag`	Effet stylé possible
PBR cubemap	`skybox/convolute + preFilterSpecular`	Trop complex pour PS1

Sun + Torch Light System

```
ourCraft stocke 2 valeurs de lumière par face :
- sunLight (4 bits, 0-15) : lumière du ciel, diminue sous terre
- torchLight (4 bits, 0-15) : lumière des sources placées

// Dans le vertex shader :
v_skyLight = max(skyLight - (15 - u_skyLightIntensity), 0);
v_ambientInt = max(v_skyLight, v_normalLight);

// u_skyLightIntensity varie dans le temps → DAY/NIGHT CYCLE !
```

> C'est comme ça qu'ourCraft fait le cycle jour/nuit : un seul uniform `u_skyLightIntensity` (15=midi, 4=nuit) qui module le skyLight de chaque face. **Très faisable pour VoxPlace** sans recalculer aucun mesh.

11c. Analyse de BetterSpades / Ace of Spades (Build & Shoot)

> Référence : github.com/xtreme8000/BetterSpades — client C/OpenGL pour Ace of Spades 0.75. Style visual : pixel art, couleurs plates, fog linéaire, face shading simple. Très proche de VoxPlace.

Couleur RGB par voxel (pas de palette)

La différence fondamentale avec Minecraft/VoxPlace : chaque bloc stocke un **uint32_t RGBA unique** (pas un index dans une palette). Le format de map `.vxl` contient la couleur exacte de chaque voxel :

```
VoxPlace                          Ace of Spades / BetterSpades
┌───┬───┬───┬───┐                 ┌───┬───┬───┬───┐
│ 7 │ 7 │ 7 │ 7 │  Même index    |#4a|#4d|#48|#4c|  Chaque bloc a
│   │   │   │   │  palette        |b3|b0|b5|b1|  un RGB unique
├───┼───┼───┼───┤                 ├───┼───┼───┼───┤
| 7 | 7 | 7 | 7 |  → couleur    |#4f|#4b|#4e|#49|  → variations
```

```
    │   │   │   │   │  uniforme    │   │   │   │   │  subtiles = fondu
    └───┴───┴───┴───┘                └───┴───┴───┴───┘
```

Le gradient de couleur (vert clair en haut des collines, sombre en bas) est créé **à la génération** du terrain côté serveur, pas dans le shader.

Face Shading — Facteurs extraits du code source

BetterSpades utilise **exactement** la même technique que VoxPlace : multiplication de la couleur RGB par un facteur fixe par direction. Voici les facteurs exacts lus dans `chunk.c` :

```
// BetterSpades — chunk.c (facteurs réels)
Face TOP    (+Y) : rgba(r * 1.000, g * 1.000, b * 1.000, 255)  // plein
soleil
Face BOTTOM (-Y) : rgba(r * 0.500, g * 0.500, b * 0.500, 255)  // très
sombre
Face NORTH  (-Z) : rgba(r * 0.875, g * 0.875, b * 0.875, 255)  // bien
éclairé
Face SOUTH  (+Z) : rgba(r * 0.625, g * 0.625, b * 0.625, 255)  // ombre
Face WEST   (-X) : rgba(r * 0.750, g * 0.750, b * 0.750, 255)  // ombre
moyenne
Face EAST   (+X) : rgba(r * 0.750, g * 0.750, b * 0.750, 255)  // ombre
moyenne
```

Comparaison avec VoxPlace :

Face	BetterSpades	VoxPlace
TOP	1.00	1.00
BOTTOM	0.50	0.50
NORTH	0.875	0.80
SOUTH	0.625	0.70
EAST	0.75	0.60
WEST	0.75	0.65

> Les facteurs sont très proches. Pas de shadow maps, pas de lumière dynamique — juste un facteur fixe par direction comme nous.

"Illumination" quand on casse un bloc

L'effet de "lumière" quand un bloc est cassé n'est **pas** un calcul de lumière. C'est du **face culling** :

```
Avant                                Après avoir cassé le bloc X
┌───┬───┐                            ┌───┐   ┐
│ A │ X │  La face entre A et X      │ A │   │  La face droite de A
```

```
    │   │   │  n'existe pas (culled)   │   │ ← │  est maintenant VISIBLE
    └───┴───┘                          └───┘   │  avec son face shading
                                                │  → ça semble "illuminé"
```

La face droite de A (factor ×0.75) apparaît soudainement. Comme toutes les faces autour du trou deviennent visibles, la zone semble plus claire. C'est un effet émergent du face culling + face shading, pas un système de lumière.

`solid_sunblock()` — Ombres diagonales (le vrai secret !)

C'est la technique qui crée l'effet de "gradient" sur les élévations. BetterSpades lance un **rayon diagonal** vers le haut depuis chaque bloc pour calculer une ombre douce :

```
// BetterSpades — chunk.c, ligne 161-173
float solid_sunblock(blocks, x, y, z) {
    int dec = 18;   // Poids décroissant (18, 16, 14, 12, ...)
    int i = 127;    // Luminosité max

    while(dec && y < map_size_y) {
        if(!isAir(blocks, x, ++y, --z))  // Monte en Y, recule en Z
            i -= dec;                     // Bloc solide → assombrit
        dec -= 2;                         // Blocs plus loin = moins
d'impact
    }
    return (float)i / 127.0F;  // 0.0 = totalement ombré, 1.0 = plein
soleil
}
```

```
  Vue de côté — le rayon va en diagonale ↗ (y+1, z-1)

          soleil ☀ (angle ~45°)
            \
             \   ← le rayon vérifie les blocs sur cette diagonale
              \
  ┌───┐   ┌───┐\┌───┐
  │   │   │ X │└\│   │  Si bloc X est solide → i -= dec
  └───┘   └───┘ │ └───┘  dec diminue à chaque pas (poids décroissant)
                │
          ┌───┐ │         Bloc courant : shade = i / 127
          │ • │ │         • reçoit une ombre partielle
          └───┘ │           car X bloque partiellement le "soleil"
                │
  ══════════════└═══════════ sol
```

La couleur finale par vertex est : `RGB × shade × face_factor × AO`

> C'est ça qui fait que les blocs sous des surplombs sont sombres et que les blocs exposés sont clairs — même sans vrai raytracing !

AO de BetterSpades vs VoxPlace

Leur AO vient du **même article** que le nôtre : 0fps.net. Formule identique :

```
// BetterSpades — chunk.c, ligne 562-569
float vertexAO(side1, side2, corner) {
    if (!side1 && !side2) return 0.25;        // Cas occluded
    return 0.75 - (!s1 + !s2 + !corner) * 0.25 + 0.25;
}

// VoxPlace — Chunk2.h
int computeVertexAO(side1, side2, corner) {
    if (side1 && side2) return 0;             // Même cas, inversé
(0=occluded)
    return 3 - (side1 + side2 + corner);      // Même logique, échelle 0-3
}
```

> ✅ **Même algorithme**, juste une échelle différente (0.25-1.0 vs 0-3 mappé par AO_CURVE).

Front-to-back Sorting (confirmé dans le code !)

```
// BetterSpades — chunk.c, ligne 144-145
qsort(chunks_draw, index, sizeof(struct chunk_render_call), chunk_sort);

// chunk_sort compare distance2D au joueur → plus proche en premier
```

Multithreading du Meshing

BetterSpades utilise `pthread_create` avec un système de **channels** (producteur/consommateur) :

1. Main thread empile des `chunk_work_packet` dans `chunk_work_queue`
2. Worker threads consomment et génèrent le mesh
3. Résultat dans `chunk_result_queue`
4. Main thread drain les résultats et upload au GPU (OpenGL = main thread only)

Le rebuild se fait en **spirale** depuis le centre (chunks proches du joueur en premier).

Ce que VoxPlace a déjà vs BetterSpades

Feature	BetterSpades	VoxPlace	Status
Face shading fixe	✅ × 0.5-1.0	✅ × 0.5-1.0	≈ Identique
Face culling	✅	✅	Identique
Fog linéaire	✅	✅	Identique
Frustum culling	✅	✅	Identique

Feature	BetterSpades	VoxPlace	Status
Per-vertex AO	✅ (même réf 0fps)	✅	Identique
Front-to-back sort	✅ `qsort`	❌	→ **prochaine étape**
`solid_sunblock` (ombres diag.)	✅	❌	**Faisable dans le shader VS**
Multithreaded meshing	✅ pthread	❌	Futur
RGB per-bloc libre	✅ uint32 RGBA	❌ (palette 32)	Palette suffit pour r/place
Greedy meshing (option)	✅	❌ (par choix)	Vertex pulling > greedy

> ✅ **VoxPlace est fonctionnellement très proche** de BetterSpades. Les 2 features manquantes les plus impactantes visuellement sont le **front-to-back sorting** (~5 lignes) et le **sunblock diagonal** (faisable dans le vertex shader en sampling la position monde).

---

12. Liens Utiles

Ressource	Lien
Voxel Wiki (Bible du voxel)	https://voxel.wiki/
Article Vertex Pulling	https://voxel.wiki/wiki/vertex-pulling/
Vidéo SSBO/Geometry	https://www.youtube.com/watch?v=IoS5opco9LA&t=1s
Tutoriels OpenGL (FR)	https://opengl.developpez.com/tutoriels/apprendre-opengl/
Pipeline diagram	https://opengl.developpez.com/tutoriels/apprendre-opengl/images/pipeline.png
Cheat sheet (SO)	https://stackoverflow.com/questions/2772570/opengl-cheat-sheet
Cheat sheet (repo)	https://github.com/henkeldi/opengl_cheatsheet
Shadertoy	https://www.shadertoy.com/
FantasyCraft Simple MinecraftClone	https://github.com/meemknight/fantasyCraft
OurCraft Huge MinecraftClone	https://github.com/meemknight/ourCraft
Veloren	https://github.com/veloren/veloren
BetterSpades (AoS 0.75 client)	https://github.com/xtreme8000/BetterSpades
OpenSpades	https://github.com/BuildandShoot/openspades
BetterSpades	https://github.com/xtreme8000/BetterSpades

Vidéos LowLevelGameDev

Sujet	Lien

Sujet	Lien
Optimisations GPU (vertex pooling, Z-prepass)	https://www.youtube.com/watch?v=FWIFQVOhIgk
Multithreading sans mutex	https://www.youtube.com/watch?v=ftHoJYvto7o
Chunk system & LODs	https://www.youtube.com/watch?v=yUUh5N2ZYHA
Architecture serveur & réseau	https://www.youtube.com/watch?v=0f0uH33X6ko

---

13. Bonnes Pratiques OpenGL

- Toujours vérifier l'init : `glfwInit()` puis `gladLoadGLLoader()`.
- Utiliser `glfwWindowHint` pour demander la bonne version de contexte (VoxPlace : 4.6 Core).
- Exclure `glad.c` pour les builds WebAssembly (le loader est fourni par Emscripten).
- Pour WASM : préférer WebGL2 et `emscripten_set_main_loop` au lieu d'une boucle `while`.
- Toujours vérifier la compilation/linkage des shaders via `glGetShaderiv` / `glGetProgramiv` et récupérer les logs avec `glGetShaderInfoLog` / `glGetProgramInfoLog`.

Dernière mise à jour : 19 février 2026